

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Components of a computer system, including CPU, memory, and I/O. Basic Operational Concepts: Instruction execution cycle, instruction fetch and decode. Bus Structures: Single-bus, multi-bus, and hierarchical buses. Factors of Performance: CPU execution time, CPI, clock cycles, and benchmarking. 8085 and 8086 Architectures: Overview, instruction sets, and addressing modes. Modern Architectures: ARM Cortex, Intel Core, and AMD Ryzen. Instruction Sets, Formats, and Addressing Modes. Number Systems: Binary, hexadecimal, and their applications in computer systems. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

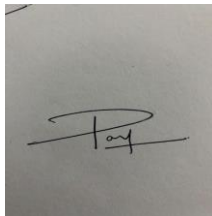
ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I (PAVAN.P/192411017) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in black ink on a light-colored surface. The signature is stylized, starting with a large 'P' and ending with a horizontal line.

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					

Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

ANSWER: -

a) Interaction between CPU, Memory, and I/O contributing to lag

- Von Neumann Bottleneck: The system suffers from the Von Neumann bottleneck. Since video feeds are massive, transferring this data continuously from I/O devices to Memory, and then to the CPU for threat detection, creates severe contention for a shared communication path.
- Bus Contention: During peak hours, the CPU demands instruction fetches, memory demands data transfers, and high-speed cameras demand I/O writes simultaneously. A shared pathway forces these components to wait (bus arbitration), introducing latency in threat detection.
- Interrupt Overhead: If the system uses programmed I/O or frequent interrupts to handle camera feeds, the CPU wastes millions of cycles context-switching instead of executing threat-detection algorithms.

b) Bus Structures: Single-Bus vs. Multi-Bus

- Single-Bus Structure: Uses a single shared pathway for CPU, Memory, and I/O. It is cheap but highly inefficient for this scenario. Only one device can communicate at a time, causing massive delays when processing high-definition video streams from multiple cameras.
- Multi-Bus Structure (e.g., Dual/Three-Bus Architecture): Dramatically enhances performance by isolating traffic.
 - A *Local Bus* connects the CPU directly to a high-speed Cache.
 - A *System Bus* connects the CPU/Cache to Main Memory.
 - An *I/O Bus* (like PCI-e) connects the cameras directly to memory via DMA (Direct Memory Access).
 - *Impact*: The CPU can fetch instructions while cameras write video frames directly to RAM concurrently, eliminating the lag.

c) Improvements in Instruction Execution Cycle

- Pipelining: Overlapping the Fetch, Decode, and Execute cycles allows the CPU to process multiple instructions simultaneously, increasing throughput (Instructions Per Clock).

- Prefetching & Caching: Storing frequently used threat-detection algorithms in L1/L2 cache reduces the need to fetch from slow main memory, shortening the execution cycle time.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

ANSWER: -

a) Impact of the Fetch–Decode–Execute Cycle on Execution Time

- Execution time is governed by the formula: $\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$.
- In a high-load trading system, the Fetch-Decode-Execute cycle is frequently stalled. When the CPU fetches a branch instruction, it cannot immediately execute the next instruction because it doesn't know which path the branch will take.
- Furthermore, frequent memory accesses mean the "Execute" phase stalls because the CPU must wait hundreds of cycles for data to arrive from RAM, artificially inflating the CPI (e.g., from an ideal of 1 to 10+).

b) Methods to Reduce CPI (Memory Access Issues)

- Memory Hierarchy (Caching): Implementing L1 and L2 caches drastically reduces the penalty for memory accesses. If stock data is in the cache, the execute phase doesn't stall, bringing CPI closer to 1.
- Data Prefetching: Hardware prefetchers can predict which stock data the CPU will need next and load it into the cache before the Fetch-Decode-Execute cycle requests it.
- Out-of-Order Execution: If a memory fetch is delayed, the CPU can look ahead in the instruction queue and execute subsequent, independent instructions, keeping the execution units busy and lowering the *effective* CPI.

c) Architectural Enhancements for Branch Instructions

- Branch Prediction: Using Branch Target Buffers (BTB) to guess the outcome of conditional branches (e.g., "if stock price > X"). Correct predictions mean the pipeline stays full.

- Speculative Execution: The CPU executes instructions from the predicted branch path before the branch is officially resolved. If correct, zero penalty; if wrong, the pipeline is flushed (but highly accurate predictors minimize this).
- Branch Delay Slots: Filling the slot after a branch instruction with a useful, independent instruction so that the pipeline doesn't stall.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

ANSWER: -

a) Influence of Addressing Modes on Data Access

- Immediate Addressing: Data is part of the instruction (e.g., **MVI A, 45H**). Used for setting fixed temperature thresholds.
- Register Addressing: Data is in a register (e.g., **MOV A, B**). Fastest execution, used for intermediate calculations.
- Direct Addressing: Instruction contains the exact memory address (e.g., **LDA 2050H**). Used if sensor logs are saved in specific RAM locations.
- Indirect Addressing: Address is pointed to by a register pair (e.g., **LDAX B**). Crucial for iterating through arrays of sensor data.

b) Possible Causes of Errors

- Confusing Immediate with I/O Direct: Using **MVI A, 02H** (loads the literal number 2) instead of **IN 02H** (reads the temperature sensor on port 2). The system will always process the number 2, ignoring the actual sensor.
- Improper Pointer Initialization: Using Indirect addressing (**MOV A, M**) without properly loading the HL register pair with the sensor data's memory address. This results in processing garbage data from an uninitialized memory location.
- Register Corruption: Overwriting a register holding a critical sensor reading before the actuator command is calculated.

c) Effective Use of 8085 Instruction Set for Reliability

- Use **IN** and **OUT** explicitly for hardware sensor/actuator interfacing.

- Use stack operations (**PUSH/POP**) to securely store register states during interrupt service routines (ISRs) to prevent data corruption.
- Utilize conditional branching (**JZ, JNC**) based on status flags after comparing sensor data (using **CMP**) against safe thresholds to trigger actuators reliably.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

ANSWER: -

a) Evaluation of Segmentation (Code, Data, Stack)

- The 8086 uses a 20-bit physical address but only has 16-bit internal registers. Segmentation solves this by using four segment registers: CS (Code), DS (Data), SS (Stack), and ES (Extra).
- Physical Address Calculation: **Physical Address = (Segment Register × 10H) + Offset**.
- *Code Segment (CS)*: Points to where instructions are fetched. *Data Segment (DS)*: Points to variables. *Stack Segment (SS)*: Points to the top of the stack for LIFO operations. This allows up to 1MB of memory to be accessed using 16-bit registers.

b) Improper Segment Handling Leading to Errors

- **Incorrect Data Access**: If a program is loaded and DS is not explicitly initialized (e.g., using **MOV AX, DATA** followed by **MOV DS, AX**), DS defaults to 0000H. The CPU will try to read program variables from the very bottom of memory, resulting in garbage data.
- **Stack Overflow**: Occurs if the SS and SP (Stack Pointer) are not set correctly, or if the stack segment overlaps with the code segment. As nested functions or interrupts push data, they eventually overwrite the executing code (CS), causing system crashes.
- **Segment Overlap**: Accidentally pointing DS and CS to the same memory area can cause a program to write data over its own instructions.

c) Managing Instructions and Addressing Modes

- **Segment Override Prefixes**: The 8086 allows default segments to be bypassed (e.g., **MOV AX, ES:[BX]**). This must be managed carefully to ensure data is fetched from the correct segment when mixing data structures.
- **Pointer Registers**: Use Base Pointers (BP) strictly for stack access (defaults to SS), and Index registers (SI, DI) strictly for array/data access (defaults to DS). Mixing their default behaviors without override prefixes causes cross-segment data corruption.

- Initialization Routine: Every program must start with a deterministic setup of CS, DS, SS, and SP before any arithmetic or logical operations occur.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

ANSWER: -

a) Importance of Number System Conversions (Hex to Binary)

- Human Readability vs. Machine Execution: Network packets are fundamentally binary (1s and 0s). Displaying raw binary to a software engineer is highly error-prone. Hexadecimal serves as a compact, human-readable shorthand where exactly one hex digit maps to exactly 4 binary bits (e.g., F = 1111).
- Protocol Debugging: Packet headers (like source/destination MAC addresses, TCP flags) are structured in byte boundaries (multiples of 8 bits / 2 hex digits). Hex allows engineers to visually inspect packet structure and memory dumps efficiently.

b) How Conversion Errors Affect Instruction Execution

- If an engineer misinterprets a hex value (e.g., reading 0xB8 as decimal 188 instead of binary 10111000), and this data is passed as a memory pointer or an opcode, it leads to catastrophic execution failures.
- Wrong OpCodes: The CPU fetches the binary pattern and executes the wrong instruction (e.g., a HALT instruction instead of an ADD).
- Segmentation Faults/Exceptions: If the misinterpreted hex value is used as a memory address, it may point to unmapped memory, triggering hardware exceptions, crashing the communication daemon, and dropping packets.

c) CPU Design and Benchmarking to Minimize Issues

- Robust CPU Exception Handling: Modern CPUs include Memory Management Units (MMUs) and exception/trap mechanisms. If an incorrect hex conversion leads to an out-of-bounds memory access, the CPU triggers a Page Fault or General Protection Fault, safely halting the errant process rather than corrupting the whole system.

- Endianness Handling: CPU architecture must clearly define and handle Endianness (Byte order). Misinterpretation often stems from Little-endian vs. Big-endian confusion in packet analysis. CPUs with bi-endian capabilities or strict software abstraction layers prevent this.
- Benchmarking & Validation: Using standardized benchmark suites (like SPEC CPU2017) combined with fuzz-testing (injecting malformed packets with manipulated hex headers) validates the CPU's pipeline and exception-handling logic. This ensures that edge cases caused by data misinterpretation are caught in simulation before deployment in real-time systems.